

Information Retrieval

Programming Assignment I

Group Name: Information Retrieval Group Project

Choudhary Asmi Aashay – 23CS30061

G. Sri Mahitha Pranavi – 23CS30021

Sanjana Kothi – 23CS10063

1 Indexing

The indexing stage takes the preprocessed collection as input. Each document in this file begins with an `.I` line containing its document identifier, followed by an `.S` line and the processed tokens. The aim is to build an inverted index that maps every distinct token to the documents in which it occurs. The input file is named `Information_Retrieval_Group_Project_processed.all`.

1.1 Reading the processed collection

The file is read sequentially rather than loading the complete collection into memory. While reading a document, its tokens are stored in a set. This removes repeated occurrences of the same token within that document, since a Boolean index only needs to record whether a term is present. When the next `.I` tag is encountered, the current document is added to the postings list of every term in its set. The final document is added after the end of the file is reached.

The parser also checks the structure of the input. It reports an error for a malformed document identifier, a repeated identifier, a missing or repeated `.S` tag, an unknown tag, or tokens occurring outside an `.S` section. These checks help prevent a damaged processed file from producing an incorrect index.

1.2 Building the postings lists

The index is represented by a dictionary whose key is a term and whose value is its postings list. Document identifiers are appended as the documents are read. The processed collection is arranged in increasing document id order, so the postings lists are produced in ascending order. A set of document identifiers is maintained separately to detect duplicates and to obtain the maximum document identifier.

Before writing the file, the vocabulary is sorted lexicographically. The first line of the index contains the vocabulary size and the maximum indexed document identifier. Every remaining line contains one term followed by its comma separated postings list. The format is therefore:

```
vocabulary_size, maximum_docid  
term docid1,docid2,docid3
```

For example, a possible entry is:

```
experimental 1,7,9,21,27
```

The generated file is named `Information_Retrieval_Group_Project_cran.index`.

1.3 Correctness checks

The completed index is read again and checked against the in-memory postings. The check verifies that the header is correct, terms are in strict lexicographic order, postings contain unique document identifiers in ascending order, every identifier is within the valid range, and the number of term lines matches the vocabulary size. The index is accepted only if all these checks pass.

1.4 Complexity

Let T be the number of processed token occurrences, V the vocabulary size, and P the total number of term–document pairs. Reading the collection and constructing the postings lists takes $O(T)$ expected time. Sorting the vocabulary takes $O(V \log V)$ time, and writing the postings takes $O(P)$ time. The overall expected time complexity is therefore $O(T + V \log V + P)$, while the index itself requires $O(P)$ space.